

CSC 3060 Tutorial 8

Project 5: part 2

Jiamu Zhao

April 21st, 23rd 2026

Overview

- Bitwise
- ReLu
- Graph
- Filter gradient

Bitwise

- This kernel applies a bitwise transformation to each pair of input elements $a[i]$ and $b[i]$ to produce $result[i]$.
- In each iteration, it first computes several basic bitwise relations like AND, OR, and XOR, then combines them with two fixed masks to form the final output.

```
const auto ua = static_cast<std::uint8_t>(a[i]);  
const auto ub = static_cast<std::uint8_t>(b[i]);  
.....
```

```
const auto shared = static_cast<std::uint8_t>(ua & ub);  
const auto either = static_cast<std::uint8_t>(ua | ub);  
const auto diff = static_cast<std::uint8_t>(ua ^ ub);  
.....
```

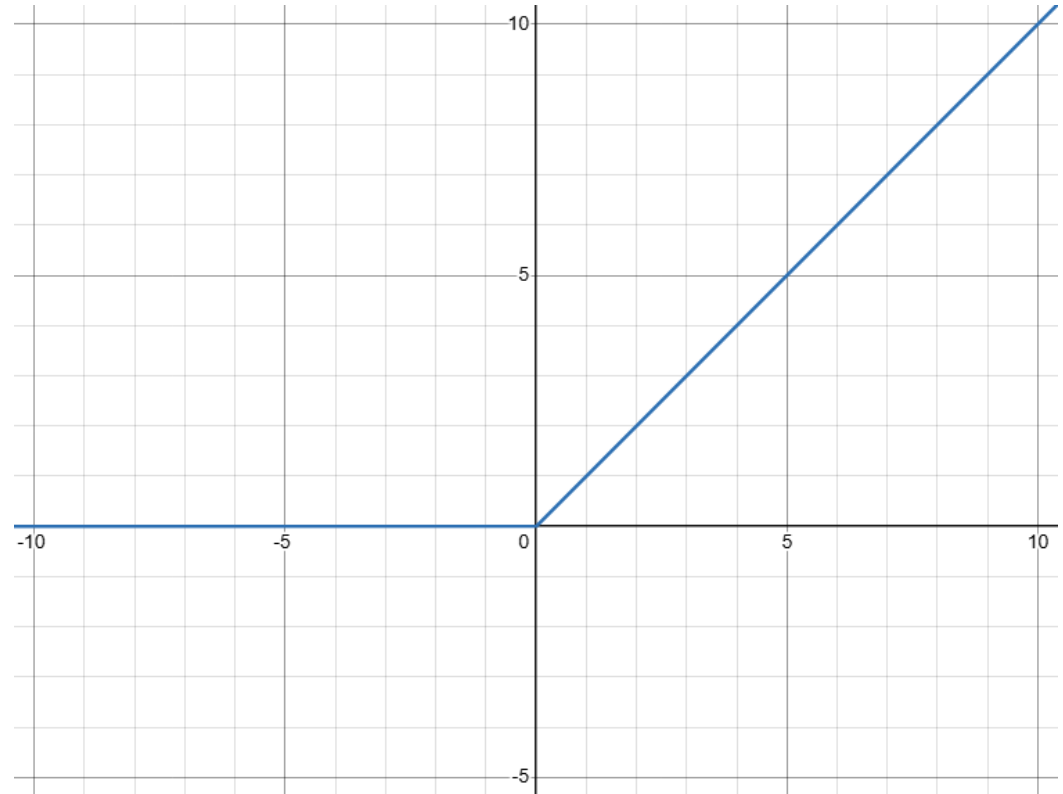
```
result[i] = static_cast<std::int8_t>(mixed0 ^ mixed1);
```

Bitwise

- **Increase data parallelism:** process multiple bytes at once using wider data types or vectorized implementations.
- 8 x int8_t -> 1 x uint64_t
- **Simplify computation:** reduce redundant intermediate operations and shorten the bitwise computation chain.
- Example: merge multiple bitwise steps into a simpler equivalent expression

ReLu

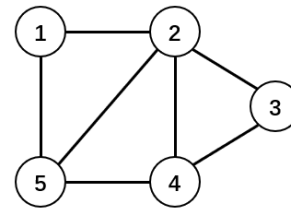
- ReLU sets negative values to 0 and keeps non-negative values unchanged.
- It is an element-wise transformation on a float array.
- **Optimization idea**
 - **Reduce branch cost**
 - Replace conditional update with a branch-free form.
 - **Preserve the same semantics**
 - Negative values become 0, others stay unchanged.



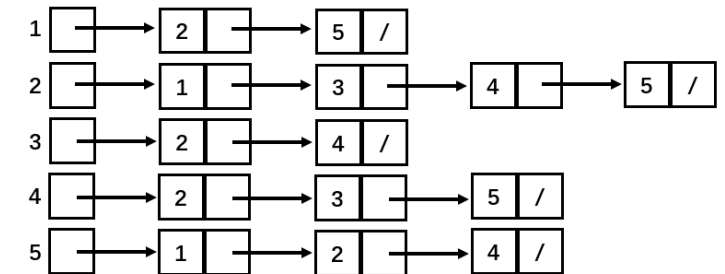
Graph

```
struct Edge {  
    int to;          // Destination node of the edge.  
    Edge* next;      // Pointer to the next edge in the adjacency list for the same node.  
};  
  
struct Node {  
    Edge* edges;      // Pointer to the list of edges connected to this node.  
};  
  
struct Graph {  
    int n;            // Number of nodes in the graph.  
    Node* nodes;      // Pointer to the array of nodes.  
};
```

- The kernel traverses all edges in the graph and accumulates a checksum.
- The naive version uses a linked-list adjacency representation.



Adjacency List of Un-directed Graph



Graph

- **Performance issue**

- Linked lists cause pointer chasing.
- Poor spatial locality hurts cache efficiency.

- **Optimization idea**

- **Improve locality** with a more contiguous layout.
- **Reduce memory overhead** by storing only needed fields.
- **Enable more regular access** for better prefetching.

```
for (int u = 0; u < graph.n; ++u) {  
    const Edge* e = graph.nodes[u].edges;  
    while (e) {  
        checksum += static_cast<std::uint64_t>(e->to);  
        e = e->next;  
    }  
}
```

Filter gradient

- The kernel processes neighboring values in a 2D grid.
- Each output step repeatedly accesses multiple fields (a to i) around a pixel location.
- Sobel Operator: It is a discrete differentiation operator used to calculate the approximate gradient of the image's gray level. The greater the gradient, the more likely it is to be an edge.

$$\mathbf{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \mathbf{A}$$

Filter gradient

- **Performance issue**

- Data is split across many separate arrays
- One computation needs values from multiple fields at the same position
- This can lead to less locality for per-pixel access

- **Optimization idea**

- Match data layout to the computation pattern
- Improve locality of related values
- Reduce scattered memory accesses
- Use layout transformation when helpful

```
struct data_struct {  
    std::vector<float> a;  
    std::vector<float> b;  
    std::vector<float> c;  
    std::vector<float> d;  
    std::vector<float> e;  
    std::vector<float> f;  
    std::vector<float> g;  
    std::vector<float> h;  
    std::vector<float> i;  
};  
  
for (int dx = -1; dx <= 1; ++dx) {  
    const std::size_t idx = row + (x + dx);  
    sum_a += data.a[idx];  
    sum_b += data.b[idx];  
    sum_c += data.c[idx];  
}
```

Thank You!

Q&A